The Emergent Geometry Engine

A substrate-agnostic curvature instrument — build log, certification, and an honest negative result

Technical report · built under a phase-gated, generator≠grader discipline · Python / Windows

In one paragraph

EGE turns any correlation/similarity matrix into a graph and measures the *shape* of that graph — discrete curvature, hyperbolicity, dimension — using those quantities as early-warning signals for when the underlying system reorganizes. The bet is reuse: one kernel serving two unrelated substrates (a quantum measurement transition; a neural network 'grokking'). Phase 0 (the kernel + an independent verifier) is built and certified to machine precision. Phase 4 (a clean head-to-head against a published competitor on the grokking task) returned a **rigorous null**: graph curvature is a useful *description* of the reorganization, but it does **not** predict it earlier than the existing spectral signal. The interesting part is the machinery that made that honest 'no' trustworthy.

1 What it is, and the core bet

The Emergent Geometry Engine takes any symmetric correlation or similarity matrix, builds a k-nearest-neighbour graph from it, and computes the discrete geometry of that graph — Ollivier / Lin–Lu–Yau and Forman–Ricci curvature, Gromov delta-hyperbolicity, and spectral dimension. Those geometric quantities are then treated as *order parameters*: signals that move when the system producing the matrix is about to reorganize.

Why graph curvature is a sensible readout: any N×N relatedness matrix is a weighted graph, and graphs carry a discrete notion of curvature that is, intuitively, *local connectivity*. It is positive where neighbours form triangles (sphere-like clusters) and strongly negative on bottleneck edges that bridge otherwise-distant regions (saddle-like). That 'negative curvature lives on bottlenecks' fact is a theorem — it underlies *over-squashing* in graph neural networks — so curvature is a cheap, local readout of how a representation is organized, and it should deflect when the representation reorganizes.

Honest posture, carried throughout. This is one shared mathematical object applied to two substrates — an analogy, not a grand unified theory. Calibrated claims only; null results are reportable and were reported.

2 The engineering discipline

Four non-negotiable rules governed everything. For a programmer this is the load-bearing part:

- **Generator ≠ grader.** The code that *computes* a quantity is never the code that *certifies* it. The verifier lives on a separate code path and is forbidden from importing the kernel — a structural guarantee, not a matter of trust. A bug cannot hide in a shared helper because there is no shared helper.
- **Evidence before claims.** Nothing asserts 'it works'; every check prints its actual number.
- **Phase-gated.** Each phase has an acceptance gate with hard numeric thresholds; no advancing until it passes.
- **Test-first.** Classic red–green TDD — watch the test fail for the *right* reason before making it pass.

3 How it was built — the orchestration

The project was constructed by an AI coding agent acting as an orchestrator that dispatched specialised sub-agents, with the anti-self-deception machinery wired in on purpose. The repeated pattern:

- **Research first, in isolation.** Before any code, separate agents verified the load-bearing claims against primary sources — exact physics formulas, library APIs, install reality on this exact machine. (This paid for itself; see Section 5.)
- **Test-first core.** The high-stakes verifier was authored red–green by the orchestrator, watched fail then pass on every module.
- **Parallel builders against fixed tests.** For independent modules the tests were written first (the spec), then builder agents were told to make them pass *without touching the tests*. A builder cannot weaken a grader it is not allowed to edit.
- **Two isolated graders at the end.** A read-only auditor re-ran everything and grepped for cheating (skipped tests, loosened tolerances, the verifier secretly importing the kernel). A *blind adversarial re-deriver*, forbidden from reading the source, reproduced the headline numbers with its own code. Both returned CERTIFIED.

4 Phase 0 — the kernel and its verifier

Phase 0 is the foundation every later phase depends on. The centrepiece is a quantum correctness benchmark: the entanglement entropy of a 1D free-fermion chain, computed **four mathematically independent ways** and required to agree to better than 1e-9:

- Peschel's correlation-matrix method (single-particle spectrum),
- brute-force exact diagonalization of the many-body Hamiltonian,
- the QuTiP library (independent third-party implementation),
- a 40-digit literature value.

They agree to ~1e-15 — machine precision. Fifteen-digit agreement between four independent derivations is effectively impossible to fake with a bug; this is the epistemic bedrock the whole project stands on. The kernel also proves a real discrete-geometry theorem on actual graphs — the discrete Lichnerowicz / Lin–Lu–Yau bound, that a Ricci-curvature lower bound forces a spectral-gap lower bound (lambda_1 >= kappa_min) — with a *negative-control* test that deliberately uses the wrong Laplacian to confirm the test would catch the single most common implementation error.

```
EMERGENT GEOMETRY ENGINE -- PHASE 0 ACCEPTANCE GATE
  [PASS] free-fermion: Peschel vs anchor          |S - anchor|       = 6.66e-16
  [PASS] free-fermion: Peschel vs independent ED  |S_Peschel - S_ED| = 1.55e-15
  [PASS] free-fermion: Peschel vs QuTiP           |Peschel - QuTiP|  = 2.11e-15
  [PASS] inequality: trace=1 / rho>=0 / S>=0 / subadditivity / SSA / Araki-Lieb
  [PASS] spectral_gap: K5_tight  lambda_1 = 1.250000000 == kappa_min  (tight)
  [PASS] spectral_gap: Q3_tight  lambda_1 = 0.666666667 == kappa_min  (tight)
  [PASS] robustness: 6 configs; ring spectral_dim mean=0.982 cov=0.075
PHASE 0 GATE: PASS  (13 passed, 0 failed)
```

Outcome: 71 tests at Phase 0 (84 once Phase 4 modules were added), gate 13/13, both isolated graders CERTIFIED.

5 Three landmines the research caught

Verifying before trusting is not ceremony. Three issues, none visible from reading code:

- **A stale dependency pin.** The brief required numpy<2.0; on this stack that pin would have *broken* scipy/scikit-learn. Removed — everything runs on numpy 2.5.
- **A citation pointing to the wrong paper.** arXiv:2605.16325, leaned on for the project's AI-safety framing, resolves to an unrelated chemistry paper. A reviewer following it would have found nothing relevant. The correct reference (arXiv:2502.17424) was substituted.
- **A library that crashes on Windows at runtime.** The standard graph-curvature package hard-codes a fork() multiprocessing call that does not exist on Windows; the install succeeds, then it dies mid-run. Curvature was re-implemented from scratch via optimal transport to avoid it.

6 Phase 4 — the grokking head-to-head

6.1 The testbed

Train a small one-layer transformer on modular addition, (a+b) mod 59, until it *groks*. Grokking is a real, well-documented oddity: the network memorizes the training set within ~100 steps (100% train accuracy, near-chance on held-out data), then — thousands of steps later, across a long dead plateau — abruptly generalizes to ~100% test accuracy. Delayed generalization with a wide gap in between, which is exactly what makes the 'does anything fire early?' question answerable.

6.2 The competitor, defined correctly

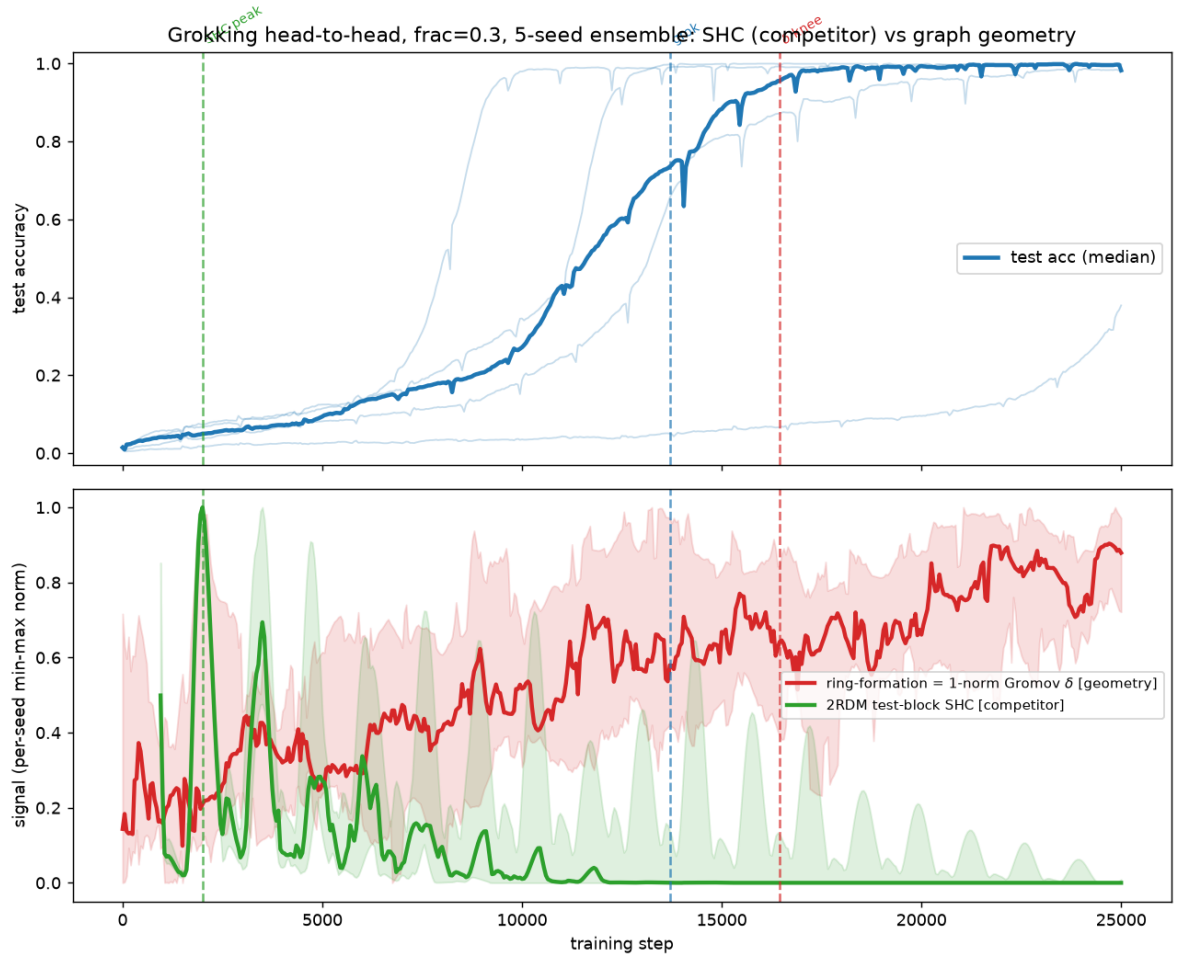
The contest: does the graph-curvature signal detect the reorganization *earlier* than a published spectral early-warning signal — the 2RDM spectral heat capacity? A research pass caught that the obvious guess at its definition was wrong. It is not a thermodynamic quantity; it is the variance of the eigenvalues of the covariance of *per-sample losses* over a sliding window of training checkpoints. Building the wrong guess would have produced a strawman competitor and a worthless comparison — so the competitor here is the genuine, verified signal.

6.3 The result (5-seed ensemble, wide-gap regime)

Across five random seeds the outcome is a clean null for the 'curvature deflects first' thesis. A positive 'lead' means the signal fires *before* the comparison event.

Metric	Median	[min, max]	Across seeds
Grok step (test acc > 0.9)	13,700	[9,100, 17,050]	4/5 grokked; 1 stalled
Spectral (SHC) peak step	2,000	[1,950, 3,500]	consistent, all 5
SHC lead over grok	+11,700	[7,150, 15,050]	positive in all 4 grokked
Geometry lead over grok	-250	[-5,900, +1,700]	straddles zero
Geometry lead over SHC	-14,450	[-18,450, -6,400]	negative in ALL 5

The spectral signal fires at step ~2,000 in every seed and leads grokking by a median **+11,700 steps**. The graph-curvature signal is roughly concurrent with generalization (it reorganizes during the *cleanup* phase) and lags the spectral signal by a median **~14,500 steps — negative in all five seeds, unanimous**. One seed did not grok within the step budget (a known seed-variance of grok timing); it is reported, not dropped.



median SHC lead over grok = 11700 steps [7150, 15050] | median geometry lead over grok = -250 steps [-5900, 1700] (<=0 = geometry lags)

Figure 1. Five-seed ensemble. Green = the spectral competitor (2RDM SHC); red = graph geometry, with across-seed bands. The spectral signal spikes early and consistently; the geometric signal is late and high-variance.

6.4 The false positive that got caught

> **The moment the pipeline policed itself**
>
> The first automated summary reported that geometry led grokking by **11,850 steps** — a headline-grade win. It was wrong. A single noisy blip in the signal had fooled the knee detector. Inspecting the raw trajectory exposed it, and the detector was rewritten to require a *durable* deflection (a commit point, not a first crossing). The pipeline converted a fake win into an honest loss. That, plus reporting the stalled seed instead of hiding it, is the entire ethos.

7 What the result means

A rigorous negative result — and the good outcome here. Three things make it trustworthy rather than merely disappointing: the competitor is the correct, *verified* signal (not a strawman, so its winning means something); the analysis caught and killed its own false positive; and the inconvenient details (the stalled seed) were reported, not swept away. Graph curvature is a useful mechanistic *description* of what grokking does to a representation — it is not an *earlier predictor* than the existing spectral signal. The project's entire architecture exists to make exactly this kind of calibrated 'no' possible: a rigorous 'no' is worth more than a flattering 'yes' you cannot trust.

8 Reproducibility and stack

Python 3.13 (Windows), numpy 2.5, scipy 1.18, networkx 3.6, scikit-learn, POT, GraphRicciCurvature, QuTiP 5.3 (optional cross-check), PyTorch 2.12 (CPU). 84 library tests. Everything reproduces from the command line:

```
.venv/Scripts/python.exe -m pytest -q                      # 84 passed
.venv/Scripts/python.exe phase0_gate.py                    # PHASE 0 GATE: PASS (13/13)
.venv/Scripts/python.exe -m experiments.grokking.run       # train + head-to-head
.venv/Scripts/python.exe -m experiments.grokking.ensemble_aggregate
```

Repository layout (kernel-first; the verifier never imports the kernel):

```
geometry/    kernel: kNN build + shortcut prune, curvature, delta, spectral dim, MDS
verify/      verifier  (separate code path; never imports geometry/)
adapters/    Adapter N (neural CKA / cosine);  Adapter Q (quantum) to come
analysis/    early-warning signals (2RDM-SHC, geometric order parameters)
experiments/grokking/   transformer + head-to-head runner + plots
tests/       84 tests          docs/   reports          phase0_gate.py
```

Phases 1–3 (an audiovisual quantum demo and the measurement-transition sweep) and Phase 5 (an AI-safety target) remain. The certified kernel they all depend on, and one decisive — if negative — experiment on top of it, are done.